

## ENGINEERING &amp; DEVOPS REFERENCE

# StockSim Pro

## A-to-Z Production Hosting Guide

A complete, step-by-step master handbook detailing architecture, Git workflows, GitHub Actions CI/CD automation, GoDaddy DNS mapping, AWS EC2 configuration, Docker orchestration, Nginx reverse proxying, and SSL encryption for **stocksimpro.in**.

TARGET DOMAIN

stocksimpro.in

INFRASTRUCTURE STACK

AWS EC2 + Docker + Nginx

DELIVERY PIPELINE

GitHub Actions CI/CD

# System Architecture & The Request Lifecycle

Deploying a full-stack financial simulator like **StockSim Pro** requires synchronizing multiple independent systems: your local code repository, cloud automation runners, DNS routing engines, server operating systems, container virtualization, and cryptographic SSL layers.

## The End-to-End Traffic Flow

When a user in Mumbai or New York opens their mobile phone or laptop and types `https://stocksimpro.in` into their browser, the following millisecond handshake takes place:



## Component Responsibilities Breakdown

| LAYER             | TECHNOLOGY                  | WHAT IT DOES                                                                                                            |
|-------------------|-----------------------------|-------------------------------------------------------------------------------------------------------------------------|
| DNS Authority     | GoDaddy                     | Translates human-readable <code>stocksimpro.in</code> into the machine's numeric IPv4 address.                          |
| Host Machine      | AWS EC2 (Ubuntu)            | A 24/7 cloud virtual computer with persistent compute power, RAM, disk, and high-speed bandwidth.                       |
| Gatekeeper        | Nginx + Certbot             | Listens on public ports 80 and 443, decrypts HTTPS certificates, and reverse-proxies requests to internal Docker ports. |
| Frontend          | React + Vite (Docker)       | Serves the trading dashboard, interactive charts, position cards, and administrative controls.                          |
| Backend API       | Node.js Express (Docker)    | Handles authentication, validates orders, queries Yahoo Finance, executes trades, and enforces rate limits.             |
| Background Engine | Protection Worker (Node.js) | Continuously monitors stock prices every 30 seconds to trigger Take Profit & Stop Loss orders automatically.            |
| Database          | MongoDB 7.0 (Docker)        | Stores user balances, holdings, transaction ledgers, leagues, and order book records across server restarts.            |

# Git & GitHub: The Code Delivery Engine

Git is a distributed version control system tracking every line of code across time. GitHub is a cloud host that acts as the single source of truth between your laptop and your production server.

## The 3 States of Code

1. **Working Directory:** The files currently sitting on your laptop hard drive where you make code edits.
2. **Staging Area (Index):** A staging staging buffer where you gather ready changes using `git add .`
3. **Local Repository (.git):** Permanent, cryptographically hashed snapshots (commits) recorded with a descriptive message.
4. **Remote Repository (origin/main):** The centralized copy stored on GitHub's cloud servers.

## Why Secrets Must NEVER Be Pushed (.gitignore)

**Critical Security Rule:** Your database credentials, JWT secrets, and Google API keys live inside `.env` files. Public or private repositories on GitHub should NEVER contain these credentials.

Your project utilizes a root `.gitignore` file that tells Git to actively ignore sensitive assets:

```
# Ignored from Git Tracking
.env
.env.*
server/.env
client/.env
node_modules/
client/dist/
```

## The Exact Git Deployment Command Sequence

```
# Step 1: Check what files have changed
git status

# Step 2: Stage all verified changes and newly added files
git add .

# Step 3: Commit the snapshot with a meaningful title
git commit -m "feat: complete UI card redesigns, order book improvements, and production readiness"

# Step 4: Push the commits up to the remote GitHub main branch
git push origin main
```

# GitHub Actions & The CI/CD Pipeline

**CI/CD** stands for **Continuous Integration & Continuous Deployment**. It automates testing, container image validation, and remote deployment without requiring you to manually SSH into servers every time you write code.

## How Your Project's Pipeline Works (.github/workflows/ci.yml)

Whenever you run `git push origin main`, GitHub automatically spins up a clean, isolated virtual machine (Ubuntu runner) and executes three sequential jobs:

```
[git push origin main]
|
▼
[JOB 1: Test & Validate Codebase]
├─ Checks out code from GitHub
├─ Installs backend & frontend dependencies (npm ci)
├─ Runs Vite production build (npm run build) to catch syntax or CSS errors
  │ (Only if Job 1 passes)
  ▼
[JOB 2: Build & Verify Docker Images]
├─ Builds the Docker backend image (stocksim-server)
├─ Builds the Docker frontend image (stocksim-client)
  │ (Only if Job 2 passes AND branch = main)
  ▼
[JOB 3: Deploy to Production Server via SSH]
├─ Securely connects to your EC2 instance using your SSH Key secret
├─ Executes: cd ~/stocksim-pro && git pull origin main
├─ Executes: docker compose up --build -d
├─ Executes: docker image prune -f (Cleans up old container layers)
```

## Configuring GitHub Repository Secrets

To allow GitHub Actions to log into your AWS EC2 server securely, add these three secrets under **GitHub Repository > Settings > Secrets and variables > Actions**:

| SECRET NAME | WHAT TO PASTE               | PURPOSE                                                                      |
|-------------|-----------------------------|------------------------------------------------------------------------------|
| VM_HOST     | 13.234.xx.xx                | The Public IPv4 address of your AWS EC2 server.                              |
| VM_USER     | ubuntu                      | The default administrative username for Ubuntu EC2 instances.                |
| VM_SSH_KEY  | Full text of your .pem file | The cryptographic private key starting with -----BEGIN RSA PRIVATE KEY-----. |

## GoDaddy DNS Configuration for stocksimpro.in

The Domain Name System (DNS) is the internet's directory. When someone types `stocksimpro.in`, DNS servers around the globe look up the corresponding public IP address of your EC2 machine.

### Required DNS Records

Inside your GoDaddy account, navigate to **Domain Portfolio > stocksimpro.in > Manage DNS**. Configure the following two records:

| TYPE  | NAME | VALUE / TARGET     | TTL                        | EXPLANATION                                                                                 |
|-------|------|--------------------|----------------------------|---------------------------------------------------------------------------------------------|
| A     | @    | YOUR_EC2_PUBLIC_IP | 1/2 Hour (or Custom: 600s) | Maps the root apex domain ( <code>stocksimpro.in</code> ) to your server's IP address.      |
| CNAME | www  | stocksimpro.in     | 1/2 Hour                   | Aliases <code>www.stocksimpro.in</code> so traffic to the www subdomain routes identically. |

**DNS Propagation Note:** DNS changes typically take between 5 to 30 minutes to propagate across worldwide recursive resolvers. You can verify propagation by running `nslookup stocksimpro.in` in your terminal.

# AWS EC2 Provisioning & Server Hardening

Amazon EC2 provides persistent cloud compute instances. For a containerized application running Node.js, Vite React, and MongoDB, a **t3.small** (2 vCPUs, 2 GB RAM) or **t3.medium** (2 vCPUs, 4 GB RAM) instance is recommended.

## Security Group (Firewall) Rules

In the AWS EC2 Management Console, you must attach a Security Group allowing the following **Inbound Rules**:

| PORT / PROTOCOL | PORT RANGE | SOURCE                      | PURPOSE                                                                     |
|-----------------|------------|-----------------------------|-----------------------------------------------------------------------------|
| SSH             | 22         | 0.0.0.0/0 (or Your Home IP) | Allows administrative terminal access and GitHub Actions CI/CD deployments. |
| HTTP            | 80         | 0.0.0.0/0                   | Allows web traffic and automated ACME verification for SSL certificates.    |
| HTTPS           | 443        | 0.0.0.0/0                   | Allows secure, encrypted web traffic for all live users.                    |

## Allocating an Elastic IP

**Why Elastic IP is mandatory:** By default, if an AWS EC2 instance is stopped and restarted, AWS assigns it a brand new public IP address. Allocating an **Elastic IP** gives your server a permanently fixed IP that never changes.

## Initial Server Configuration Commands

Log in from your computer via SSH and install the required tools:

```
# Connect to EC2
ssh -i "stocksim-key.pem" ubuntu@YOUR_EC2_PUBLIC_IP

# Update system repositories
sudo apt update && sudo apt upgrade -y

# Install Docker engine, Docker Compose v2, Git, and Nginx
sudo apt install -y docker.io docker-compose-v2 git nginx certbot python3-certbot-nginx

# Allow the ubuntu user to run Docker without sudo
sudo usermod -aG docker ubuntu
newgrp docker
```

## Docker & Docker Compose Architecture

Docker packages code and all its dependencies into portable, isolated containers. This eliminates environment drift ("it worked on my machine, but failed on the server").

### The Multi-Container Topology

Your project uses **docker-compose.yml** to launch three networked microservices:

```
graph LR
    subgraph "stocksim-network (Bridge)"
        direction TB
        subgraph "stocksim-client"
            direction TB
            proxy["proxy /api"]
            port["(Port 8080 ← 80)"]
        end
        subgraph "stocksim-server"
            direction TB
            server["[stocksim-server]"]
        end
        subgraph "stocksim-mongo"
            direction TB
            mongo["mongo:27017"]
            mongo_data["(Volume: mongo-data)"]
        end
        proxy --> server
        mongo --- mongo_data
    end
```

### Essential Docker Operations

```
# Build and start all 3 containers in background mode
docker compose up -d --build

# View currently running containers and health status
docker ps

# Stream live real-time server logs
docker logs -f stocksim-server

# View MongoDB container logs
docker logs -f stocksim-mongo

# Stop all services safely without losing database data
docker compose down
```

## Nginx Reverse Proxy & Free Let's Encrypt SSL

Nginx acts as the front door of your server. It terminates HTTPS encryption on port 443 and distributes requests internally to your Docker containers.

### Nginx Configuration File (/etc/nginx/sites-available/stocksimpro.in)

```
server {
    server_name stocksimpro.in www.stocksimpro.in;

    # 1. Frontend: Route root web traffic to React Docker Container
    location / {
        proxy_pass http://localhost:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # 2. Backend API: Route /api requests to Express Docker Container
    location /api/ {
        proxy_pass http://localhost:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

### Activating the Site & Generating Free SSL Certificates

```
# Enable site configuration
sudo ln -s /etc/nginx/sites-available/stocksimpro.in /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default

# Test Nginx syntax
sudo nginx -t

# Reload Nginx
sudo systemctl reload nginx

# Generate SSL certificate using Certbot
sudo certbot --nginx -d stocksimpro.in -d www.stocksimpro.in
```

**Automated Renewal:** Certbot installs an automatic systemd timer ( `certbot.timer` ) that auto-renews your SSL certificate every 60 days without requiring manual intervention.

# The Production Operations Runbook

A quick-reference cheat sheet for day-to-day operations, updates, and disaster recovery.

## Daily Updates Workflow (Push to Live)

```
# On your local laptop:
git add .
git commit -m "feat: new feature description"
git push origin main

# That's it! GitHub Actions handles testing, building, and deploying automatically.
```

## Manual Server Deployment (If CI/CD is bypassed)

```
# 1. SSH into the server
ssh -i "stocksim-key.pem" ubuntu@YOUR_EC2_PUBLIC_IP

# 2. Navigate to project directory
cd ~/stocksim-pro

# 3. Pull latest code from GitHub
git pull origin main

# 4. Rebuild and restart containers
docker compose up -d --build
```

## Emergency Troubleshooting Cheat Sheet

| SYMPTOM                              | DIAGNOSTIC COMMAND                         | FIX                                                                                                        |
|--------------------------------------|--------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Website returns 502<br>Bad Gateway   | <code>docker ps</code>                     | A Docker container crashed. Run <code>docker compose up -d</code> to bring it back.                        |
| CORS error in browser<br>console     | <code>nano server/.env</code>              | Verify <code>CLIENT_ORIGIN</code> includes <code>https://stocksimpro.in</code> .                           |
| SSL Certificate<br>expired / invalid | <code>sudo certbot<br/>certificates</code> | Run <code>sudo certbot renew --force-renewal</code> followed by <code>sudo systemctl reload nginx</code> . |
| Disk space full on EC2               | <code>df -h</code>                         | Prune unused Docker layers using <code>docker system prune -a --volumes</code> .                           |